

11.1 Allocating Memory with `malloc`

[This section corresponds to parts of K&R Secs. 5.4, 5.6, 6.5, and 7.8.5]

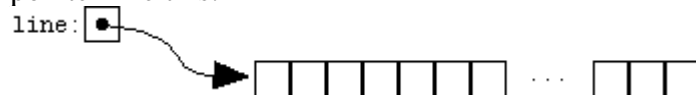
A problem with many simple programs, including in particular little teaching programs such as we've been writing so far, is that they tend to use fixed-size arrays which may or may not be big enough. We have an array of 100 `ints` for the numbers which the user enters and wishes to find the average of--what if the user enters 101 numbers? We have an array of 100 `chars` which we pass to `getline` to receive the user's input--what if the user types a line of 200 characters? If we're lucky, the relevant parts of the program check how much of an array they've used, and print an error message or otherwise gracefully abort before overflowing the array. If we're not so lucky, a program may sail off the end of an array, overwriting other data and behaving quite badly. In either case, the user doesn't get his job done. How can we avoid the restrictions of fixed-size arrays?

The answers all involve the standard library function `malloc`. Very simply, `malloc` returns a pointer to n bytes of memory which we can do anything we want to with. If we didn't want to read a line of input into a fixed-size array, we could use `malloc`, instead. Here's the first step:

```
#include <stdlib.h>

char *line;
int linelen = 100;
line = malloc(linelen);
/* incomplete -- malloc's return value not checked */
getline(line, linelen);
```

`malloc` is declared in `<stdlib.h>`, so we `#include` that header in any program that calls `malloc`. A "byte" in C is, by definition, an amount of storage suitable for storing one character, so the above invocation of `malloc` gives us exactly as many `chars` as we ask for. We could illustrate the resulting pointer like this:



The 100 bytes of memory (not all of which are shown) pointed to by `line` are those allocated by `malloc`. (They are brand-new memory, conceptually a bit different from the memory which the compiler arranges to have allocated automatically for our conventional variables. The 100 boxes in the figure don't have a name next to them, because they're not storage for a variable we've declared.)

As a second example, we might have occasion to allocate a piece of memory, and to copy a string into it with `strcpy`:

```
char *p = malloc(15);
/* incomplete -- malloc's return value not checked */
strcpy(p, "Hello, world!");
```

When copying strings, remember that all strings have a terminating `\0` character. If you use `strlen` to count the characters in a string for you, that count will *not* include the trailing `\0`, so you must add one before calling `malloc`:

```
char *somesstring, *copy;
...
copy = malloc(strlen(somesstring) + 1);          /* +1 for \0 */
/* incomplete -- malloc's return value not checked */
strcpy(copy, somesstring);
```

What if we're not allocating characters, but integers? If we want to allocate 100 `ints`, how many bytes is that? If we know how big `ints` are on our machine (i.e. depending on whether we're using a 16- or 32-bit machine) we could try to compute it ourselves, but it's much safer and more portable to let C compute it for us. C has a `sizeof` operator, which computes the size, in bytes, of a variable or type. It's just what we need when calling `malloc`. To allocate space for 100 `ints`, we could call

```
int *ip = malloc(100 * sizeof(int));
```

The use of the `sizeof` operator tends to look like a function call, but it's really an operator, and it does its work at compile time.

Since we can use array indexing syntax on pointers, we can treat a pointer variable after a call to `malloc` almost exactly as if it were an array. In particular, after the above call to `malloc` initializes `ip` to point at storage for 100 `ints`, we can access `ip[0]`, `ip[1]`, ... up to `ip[99]`. This way, we can get the effect of an array even if we don't know until run time how big the ``array" should be. (In a later section we'll see how we might deal with the case where we're not even sure at the point we begin using it how big an ``array" will eventually have to be.)

Our examples so far have all had a significant omission: they have not checked `malloc`'s return value. Obviously, no real computer has an infinite amount of memory available, so there is no guarantee that `malloc` will be able to give us as much memory as we ask for. If we call `malloc(100000000)`, or if we call `malloc(10)` 10,000,000 times, we're probably going to run out of memory.

When `malloc` is unable to allocate the requested memory, it returns a *null pointer*. A null pointer, remember, points definitively nowhere. It's a ``not a pointer" marker; it's not a pointer you can use. (As we said in section 9.4, a null pointer can be used as a failure return from a function that returns pointers, and `malloc` is a perfect example.) Therefore, whenever you call `malloc`, it's vital to check the returned pointer before using it! If you call `malloc`, and it returns a null pointer, and you go off and use that null pointer as if it pointed somewhere, your program probably won't last long. Instead, a program should immediately check for a null pointer, and if it receives one, it should at the very least print an error message and exit, or perhaps figure out some way of proceeding without the memory it asked for. But it cannot go on to use the null pointer it got back from `malloc` in any way, because that null pointer by definition points nowhere. (``It cannot use a null pointer in any way" means that the program cannot use the `*` or `[]` operators on such a pointer value, or pass it to any function that expects a valid pointer.)

A call to `malloc`, with an error check, typically looks something like this:

```
int *ip = malloc(100 * sizeof(int));
if (ip == NULL)
{
    printf("out of memory\n");
    exit or return
}
```

After printing the error message, this code should return to its caller, or exit from the program entirely; it cannot proceed with the code that would have used `ip`.

Of course, in our examples so far, we've still limited ourselves to ``fixed size" regions of memory, because we've been calling `malloc` with fixed arguments like 10 or 100. (Our call to `getline` is still limited to 100-character lines, or whatever number we set the `linelen` variable to; our `ip` variable still points at only 100 `ints`.) However, since the sizes are now values which can in principle be determined at run-time, we've at least moved beyond having to recompile the program (with a bigger

array) to accommodate longer lines, and with a little more work, we could arrange that the ``arrays" automatically grew to be as large as required. (For example, we could write something like `getline` which could read the longest input line actually seen.) We'll begin to explore this possibility in a later section.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)